Ask the Machines

NASA Space Apps Challenge 2026 - Project Plan and Context

Purpose of this document: This is a full summary of the planning done so far for this project. It is written so an AI coding assistant can read it and help build the project consistently. Items marked **DECIDED** are confirmed by the developer. Items marked **SUGGESTED** are recommendations that are not final yet and can be changed.

1. The Challenge

Challenge 09: **Abandoned but not Forgotten: Storytelling about NASA's Discarded Equipment on the Moon and Mars.** Since the 1960s NASA has left hardware across the solar system: rovers and instruments that finished their science missions, and probes still traveling away from Earth. The task is to tell the story of some or all of this equipment in a way that introduces **school-age space enthusiasts** to the hardware and the science it made possible.

2. Concept

DECIDED: A scroll-driven storytelling website where the scrollbar acts as a timeline. As the user scrolls, a machine is shown leaving Earth and traveling to where it now rests (Moon, Mars, or deep space). Each machine has a story panel (its mission and the science it made possible) and a chatbot persona that kids can talk to in first person.

- Audience: school-age kids, so language must be simple, friendly, and accurate.
- Landing page shows Earth and NASA Headquarters, and introduces the first machine's mission.
- Scrolling moves the first machine away from Earth along its path; on arrival it parks and its story appears.
- Scrolling on brings the next machine, in chronological order. Plan for 3 to 4 machines in total.
- A side timeline bar with year markers shows progress.

Candidate machines (not final, to be chosen with the teammate): Voyager 1 (deep space), Apollo lunar rover or lander (Moon), Spirit or Opportunity (Mars rovers), Ingenuity (Mars helicopter). Order chronologically by year.

3. Team and Roles

Two people. **DECIDED:**

- **Developer (user):** builds the website (Angular app, scroll animation, layout, chat UI).
- **Teammate:** handles everything related to the AI chatbot (personas, prompts, fact grounding, backend endpoint).
- **Both:** research the facts, choose the machines, and design the look.

Developer's strongest skill is Angular web development. Timeline: roughly **3 weeks**.

4. Technical Decisions

Topic	Decision	Status
Framework	Angular, standalone components, plain SPA	DECIDED
Rendering	No SSR. Prerendering (SSG) only optionally at the very end	DECIDED
2D vs 3D	2D. Earth rotation faked with a scrolling texture inside a circular mask	DECIDED
Animation	GSAP with ScrollTrigger (scroll progress drives the scene)	DECIDED
Three.js	Not used for now. Optional later for one 3D model if time allows	SUGGESTED
Chat backend	Serverless function (Netlify or Vercel) that hides the API key	SUGGESTED
Hosting	Netlify or Vercel; deploy a blank page early	SUGGESTED

How the 2D rotating Earth works: Use one wide flat Earth map image (equirectangular, 2K resolution) placed inside a circular mask (CSS border-radius 50% with overflow hidden). Animate its horizontal position (GSAP, or CSS background-position) tied to scroll. Overlay a shading gradient (darker toward one edge) and an atmosphere glow so it reads as a sphere. Optional: a few small lights on the dark side.

Angular and GSAP notes: Register ScrollTrigger once. Run GSAP setup outside Angular's zone (NgZone.runOutsideAngular) for smooth scrolling. Kill ScrollTriggers and tweens when components are destroyed (DestroyRef or ngOnDestroy). Keep animations to simple transforms (move, rotate, scale, opacity) so it runs well on phones. Test mobile early, since scroll pinning behaves differently there.

5. Page Flow

```
1. LANDING        Earth + NASA HQ, headline, intro to first machine's mission
2. SCROLL: LAUNCH Machine leaves Earth and follows its path (scroll-scrubbed)
3. ARRIVAL        Machine parks at Moon / Mars / deep space.
                  Story panel appears. Chat bubble shows that machine's avatar.
4. SCROLL ON      Next machine takes over (chronological). Repeat.
Side timeline bar: progress + year markers throughout.
```

Landing page design (mockup agreed): dark space background with stars. Navigation at the top (logo text 'ASK THE MACHINES', links, Chat button). Left side: small label 'NASA SPACE APPS 2026', headline 'Abandoned, not forgotten.' (second line in yellow), a two-line subtitle, and a yellow 'Begin the journey' button. Large lit Earth in the lower right with an atmosphere glow. NASA Headquarters as a small building on the Earth's horizon with a label. The first machine on the horizon with a dotted trajectory toward the Moon. A vertical timeline bar on the right edge. Palette: deep navy background, bright blue accents, yellow highlights, white text.

6. Angular Structure

Rule of thumb: if it draws something on screen it is a COMPONENT. If it holds data, logic, or talks to the outside world it is a SERVICE. **DECIDED:** use ONE reusable machine-section component driven by data (not one component per machine).

```
src/
  app/
    app.ts (or app.component.ts)  COMPONENT - root; layers stage behind landing
    app.config.ts                 CONFIG    - provideHttpClient etc.

    components/
```

```
      landing/          COMPONENT - hero text, 'Begin the journey' button, chat teaser
      stage/            COMPONENT - pinned full-screen scene: Earth, path, machine sprite
      machine-section/  COMPONENT - one scroll section; takes a Machine as input
      story-panel/      COMPONENT - story text + science facts (inside machine-section)
      timeline-bar/     COMPONENT - side progress bar with year markers
      chat-bubble/      COMPONENT - floating button, lower right, shows current avatar
      chat-panel/       COMPONENT - chat window that opens from the bubble

    services/
      machine-data.service.ts     SERVICE - supplies the machine list (reads machines.json)
      scroll-timeline.service.ts  SERVICE - GSAP + ScrollTrigger setup; exposes
                                            activeMachine and progress signals
      chat.service.ts             SERVICE - HTTP POST to backend; keeps chat history
                                            per machine (map keyed by machineId)

    models/
      machine.model.ts        INTERFACE (not a service)
      chat-message.model.ts   INTERFACE (not a service)

  assets/
    data/machines.json   story text, years, facts, avatar paths per machine
    img/                 earth-2k.jpg, nasa-hq.svg, machine sprites, avatars

netlify/functions/chat.ts   BACKEND (outside Angular; holds the API key)
```

- **stage** is a pure visual component. It reads the progress and activeMachine signals and moves the sprite. It holds no data. **DECIDED/SUGGESTED:** Earth and NASA HQ live in **stage** (not landing) so Earth stays on screen and the launch starts from it. Landing is only text and buttons on top.
- **scroll-timeline.service** is the brain of the animation (not needed for the very first landing build, add it when starting the launch animation).
- **machine-section** is repeated with @for over the machine list; each is a tall scroll area that triggers the story panel on arrival.

Machine model

```
export interface Machine {
  id: string;                 // 'voyager-1'
  name: string;
  year: number;
  destination: 'moon' | 'mars' | 'deep-space';
  story: string;
  facts: string[];
  sprite: string;             // path in assets/img
  avatar: string;             // chat bubble icon
  greeting: string;           // first chat message
  suggestedQuestions: string[];
}
```

7. Chatbot Design

DECIDED: The chatbot is a **floating bubble icon in the lower right corner**. The icon's avatar is different depending on which machine's story is currently in view, and when the icon changes, the chatbot (persona) changes with it.

- chat-bubble reads the activeMachine signal from scroll-timeline.service and swaps its avatar (crossfade or flip, not an instant swap).

- chat.service sends the machineId with every message so the backend selects the right persona and fact sheet.
- Keep a separate chat history per machine, so scrolling back restores that earlier conversation.

API contract with the teammate (keep fixed so both can work independently):

```
POST /chat
Request:  { "machineId": "voyager-1", "message": "Are you still working?" }
Response: { "reply": "..." }
```

SUGGESTED (not final):

- When the user scrolls to a new machine while chat is open: close the panel and animate the bubble to the new avatar with a small 'new' dot. (Alternative: keep it open with a divider like 'Now talking to Spirit'.)
- Before any machine is active (landing page), show a generic NASA or mission-control avatar, or hide the bubble until the first machine is in view.
- Add 2 to 3 tappable suggested questions per machine, which helps younger users.
- Ground the chatbot in a fact sheet per machine taken from NASA mission pages so it does not invent details, and keep replies age-appropriate.
- Never call the AI API directly from Angular, because that exposes the API key. Use the serverless function.

8. Prerequisites

Tools to install

- Node.js (current LTS), which includes npm
- Angular CLI: npm install -g @angular/cli
- Git, plus a shared GitHub repository
- VS Code

Project setup

- ng new (plain SPA, NO SSR, standalone components, CSS or SCSS)
- npm install gsap (ScrollTrigger is included; register it in code). Do NOT install three for now.
- Accounts: GitHub, and Netlify or Vercel (hosting and the later serverless chat function)

Assets to download for the landing page

- Earth texture: one wide flat world map (equirectangular or Blue Marble style), 2K size. Sources: NASA Visible Earth or Solar System Scope textures (check attribution rules).
- NASA HQ: a simple illustration or photo with a transparent background (PNG or SVG); a basic self-drawn one is fine.
- First machine sprite (PNG or SVG), once the first machine is chosen with the teammate.
- One or two Google Fonts: a bold one for headlines and a clean one for body text.
- Stars can be made with CSS or a small PNG; do not spend time hunting for them.

- Check licenses for non-NASA images. NASA media is generally free to use, but do not imply NASA endorsement. Credit CC-BY authors.

Save assets in src/assets/img/ with clear names such as earth-2k.jpg and nasa-hq.svg.

9. Build Order (first steps) and 3-Week Plan

Immediate order of work (landing page first)

- **Step 1 - Setup (about half a day):** ng new, install gsap, generate only landing and stage, set global dark background and fonts. Deploy a blank page early.
- **Step 2 - Stage with 2D Earth (1 to 2 days):** circular masked Earth with a scrolling texture, shading gradient, and glow. Fill the screen behind everything and resize properly.
- **Step 3 - Landing text over the stage (half a day):** layer landing on top in the root component. Hardcode the headline, subtitle, and button text.
- **Step 4 - Add the first machine sprite (1 day):** place it near Earth; it does not move yet.
- **Step 5 - First scroll interaction (1 to 2 days):** Earth rotation and/or launch movement using GSAP ScrollTrigger.

Needed now: landing and stage components, the root component, assets, and global styles. **Skip for now:** machine-section, story-panel, timeline-bar, chat-bubble, chat-panel, chat.service, machine-data.service, the models, and the backend.

Rough 3-week plan

- **Week 1:** setup, stage with 2D Earth, landing text, first sprite, first scroll interaction.
- **Week 2:** first machine's full journey, story panel, timeline bar, machine data and models.
- **Week 3:** remaining machines, chat bubble and panel connected to the teammate's backend, polish, mobile testing, demo video.

In parallel with the teammate: agree on the POST /chat contract and pick the first 3 to 4 machines early.

Demo tip: record a short video of a kid-style conversation with a machine; it is a strong way to show the project.

10. Instructions for the AI Coding Assistant

- Follow the Angular structure in section 6 exactly, and keep components and services clearly separated.
- Use standalone components and Angular signals for shared state (activeMachine, progress).
- Do not add SSR and do not install Three.js unless the developer asks.
- Build only what the current step needs. Start with the landing and stage components (section 9) and ask before adding other pieces.
- Keep GSAP outside Angular's zone and clean up ScrollTriggers on destroy.
- Keep text and content kid-friendly and factually accurate.
- When a decision is marked SUGGESTED, ask the developer before treating it as final.